Comparing the Performance of DQN and PPO for Training Pong: A Study on Efficiency and Convergence Speed

John Vernon B. Baldeo
College of Computing and Information Technology (CCIT)
National University - Manila
Email: baldeojb@students.national-u.edu.ph

Earl Anthony Z. Gamboa
College of Computing and Information Technology (CCIT)
National University - Manila
Email: gamboaez@students.national-u.edu.ph

Rainnand P. Montaniel
College of Computing and Information Technology (CCIT)
National University - Manila
Email: montanielrp@students.national-u.edu.ph

***Abstract*—This study presents a comparative analysis between two widely used reinforcement learning algorithms Deep Q-Network (DQN) and Proximal Policy Optimization (PPO) in training an agent to play the Atari game Pong. The focus is on determining which algorithm provides better training efficiency and faster convergence toward optimal gameplay performance. Both models are trained under similar hyperparameter constraints and evaluated based on training stability, average reward progression, and time to reach target performance. Experimental results show that while DQN tends to converge faster during early training stages, PPO demonstrates greater stability and smoother learning behavior in the long run. The findings highlight the trade-offs between value-based and policy-based reinforcement learning methods in environments with discrete, low-dimensional action spaces such as Pong.**

***Index Terms*—Reinforcement Learning, Deep Q-Network, Proximal Policy Optimization, Pong, Atari, Training Efficiency, Convergence Speed**

I. Introduction

Reinforcement Learning (RL) has become a cornerstone in the field of artificial intelligence, allowing agents to learn optimal behavior through interactions with their environments. Among the benchmark environments, the Atari game Pong is a standard testing ground for evaluating the performance of RL algorithms due to its simple dynamics yet competitive learning challenge.

This research focuses on comparing the performance of two major reinforcement learning paradigms: the value-based Deep Q-Network (DQN) and the policy-based Proximal Policy Optimization (PPO). Both methods have shown strong results in playing Atari games, but they differ in how they approach learning. DQN learns a value function that estimates the expected future reward for each action, while PPO directly optimizes a policy that outputs actions given states.

The goal of this study is to determine which of these algorithms provides better and faster training when applied to Pong. By analyzing metrics such as convergence rate, training stability, and average episodic reward, this work aims to contribute to a deeper understanding of algorithmic efficiency in reinforcement learning environments.

II. Literature Review

Several studies have applied reinforcement learning (RL) techniques to develop intelligent agents capable of mastering complex control tasks, particularly within classic Atari environments such as Pong. Early work by Mnih et al. [1], [2] introduced the Deep Q-Network (DQN), which enabled agents to learn directly from raw pixel inputs using convolutional neural networks. This approach achieved near-human performance on multiple Atari games (including Pong) and marked a significant milestone in deep reinforcement learning (DRL).

Following the success of DQN, various extensions were proposed to address its inherent limitations. Double DQN, developed by van Hasselt et al. [3], mitigated the overestimation of Q-values that often led to unstable learning. Similarly, Dueling DQN by Wang et al. [4] enhanced learning efficiency by decomposing the Q-function into separate value and advantage estimations, allowing better generalization across similar actions. The Rainbow DQN architecture by Hessel et al. [5] further combined several improvements such as prioritized experience replay, multi-step returns, and noisy networks achieving state-of-the-art performance across multiple Atari benchmarks, including Pong.

Actor-critic and policy gradient algorithms have also demonstrated remarkable progress in RL game environments. The Asynchronous Advantage Actor-Critic (A3C) algorithm, introduced by Mnih et al. [6], leveraged multiple asynchronous agents to stabilize training and speed up convergence compared to single-threaded DQN models. In contrast, Proximal Policy Optimization (PPO), proposed by Schulman et al. [7], provided a simplified yet robust policy-gradient method that

maintained high performance and ease of implementation across diverse environments. These algorithms are categorized as model-free, policy-based methods and are known for their adaptability, though they typically require more samples than value-based approaches.

A. Deep Q-Network (DQN)

DQN is a value-based algorithm that combines Q-learning with deep neural networks. Its architecture typically consists of three convolutional layers followed by two fully connected layers that output Q-values corresponding to possible actions. The input is usually a stack of four consecutive grayscale frames (84×84 pixels each), which enables temporal awareness of the ball and paddle movement.

Key parameters include:

- Learning rate: typically between 10^{-4} and 10^{-5}
- Discount factor (γ): 0.99
- Replay buffer size: 10^5 to 10^6
- Batch size: 32
- Exploration strategy: ϵ-greedy with ϵ decay

Evaluation of DQN on Pong has shown rapid improvement in the early stages of training, achieving competent gameplay within 1–2 million frames. However, DQN can suffer from Q-value overestimation and instability, especially when the learning rate or replay buffer sampling is not well tuned.

B. Proximal Policy Optimization (PPO)

PPO is a policy-based method under the actor-critic framework. It uses two networks an actor that outputs a probability distribution over actions and a critic that estimates the value function. PPO introduces a clipped surrogate objective to constrain policy updates, ensuring that new policies do not deviate excessively from the old ones.

Typical architectural design for PPO in Pong mirrors that of DQN, using shared convolutional layers followed by separate fully connected branches for policy (actor) and value (critic). Common hyperparameters include:

- Learning rate: 2.5×10^{-4}
- Discount factor (γ): 0.99
- Clipping parameter (ϵ): 0.1 to 0.2
- GAE (λ): 0.95
- Epochs per update: 3–10

PPO's main advantage lies in its training stability and robustness to hyperparameter variations. Studies such as Schulman et al. [7] demonstrated that PPO achieves comparable or superior results to A3C and TRPO with simpler implementation and reduced variance in reward progression.

C. Comparison of DQN and PPO in Pong

While both algorithms have achieved strong performance in Pong, they differ significantly in their learning dynamics. DQN, being off-policy and value-based, tends to learn faster in the early phases due to direct Q-value updates and efficient replay memory reuse. However, it can be unstable when the target Q-values diverge or when replay samples introduce outdated experiences.

PPO, on the other hand, is an on-policy method that trades sample efficiency for stability. It requires more interactions with the environment but produces smoother reward curves and better long-term consistency. In terms of computation, PPO generally demands more processing time per iteration due to multiple policy updates per batch.

Empirical comparisons from Henderson *et al.* [8] and other works suggest that for simple games like Pong, DQN converges faster but exhibits more variance, while PPO demonstrates slower yet steadier convergence. The trade-off between speed and stability forms the foundation of this research's experimental focus.

III. Methodology

A. Environment Setup

The Atari Pong environment from the Gymnasium library, built upon the Atari Learning Environment (ALE), was employed as the experimental platform. The study selected `PongNoFrameskip-v4` as the environment with the following actions:

TABLE I
ACTION SPACE OF PONGNOFRAMESKIP-V4

Action ID	Description
0	No operation
1	Game Start
2	Move paddle up
3	Move paddle down
4	Move paddle up
5	Move paddle down

The environment also provided pixel-based state observations at a resolution of 84×84 in grayscale format. To capture temporal dependencies such as ball motion and paddle dynamics, four consecutive frames were stacked to form each state. The agent received a reward of $+1$ for scoring, -1 for conceding a point, and zero otherwise with a maximum/minimum reward of +21/-21. Frame-skipping of four was applied to reduce computational load and smooth transitions between states.

To ensure reproducibility, random seeds were fixed across PyTorch, NumPy, and Gymnasium. All experiments were executed on GPU-enabled hardware to guarantee consistent computational performance and training speed. Environment normalization was handled using Stable-Baselines3's `VecNormalize` wrapper, ensuring consistent observation scaling and reward clipping. The full software configuration was standardized using Python 3.10, PyTorch 2.3, Stable-Baselines3 2.2, Gymnasium 0.29, and ALE-py 0.8.1.

Training logs, episode rewards, and loss metrics were recorded through TensorBoard via Stable-Baselines3's integrated logging module. Evaluation runs were performed after every 100 episodes to monitor policy progression and stability. All configuration files, source code, and checkpoints were version-controlled to ensure transparency and replicability of results.

To ensure parity in comparison, both algorithms used identical preprocessing, normalization, and observation pipelines. Frame skipping was set to four frames per action, and vectorized environments were implemented using `DummyVecEnv` and `VecFrameStack` to handle parallel simulations. The DQN model was trained on four parallel environments (`n_envs=4`), while PPO used eight (`n_envs=8`) to improve on-policy sample collection. Both environments were seeded (`seed=100`) to guarantee reproducibility.

B. Neural Network Architecture

Both DQN and PPO employed a shared convolutional neural network (CNN) architecture defined as a custom feature extractor inherited from `BaseFeaturesExtractor` in Stable Baselines3. The CNN processed the four stacked input frames to generate a 512-dimensional latent feature vector. The architecture consisted of the following layers:

- **Conv Layer 1:** 32 filters, kernel size 8×8, stride 4, activation ReLU
- **Conv Layer 2:** 64 filters, kernel size 4×4, stride 2, activation ReLU
- **Conv Layer 3:** 64 filters, kernel size 3×3, stride 1, activation ReLU
- **Flatten:** Converts feature maps into a 1D vector
- **Fully Connected:** Linear layer with 512 units and ReLU activation

This CNN feature extractor was shared across both agents to maintain uniform visual feature representation. For DQN, the extracted features were passed into fully connected layers that output Q-values corresponding to each action. For PPO, the same CNN backbone was shared between the actor and critic networks, branching into separate policy and value estimation heads.

C. Algorithm Implementation

Two agents were implemented using Stable Baselines3:

- **DQN Agent:** Followed an off-policy value-based learning framework with experience replay and target network updates every 1000 steps. Exploration was managed via an ϵ-greedy strategy, decaying ϵ from 1.0 to 0.01 over 10% of total timesteps. Core hyperparameters included a learning rate of 1×10^{-4}, discount factor $\gamma = 0.99$, batch size of 32, and replay buffer size of 10,000 transitions. The model began training after a warm-up phase of 100,000 steps.
- **PPO Agent:** Implemented as an on-policy actor-critic model using clipped policy optimization for stability. Hyperparameters included a learning rate of 2.5×10^{-4}, discount factor $\gamma = 0.99$, clipping parameter $\epsilon = 0.1$, entropy coefficient 0.01, generalized advantage estimation (GAE) $\lambda = 0.9$, and four epochs per update. Each policy update was based on 128 rollout steps and a batch size of 32.

Both models used the same CNN extractor and were trained for five million timesteps. Training progress and model checkpoints were logged and saved at fixed intervals using callback functions.

D. Training Configuration

All experiments were executed with GPU acceleration to ensure efficient computation. Checkpoints were saved every 25,000 steps, and training metrics such as episodic rewards, losses, and episode lengths were logged using a custom callback class (`RewardLoggingCallback`). The callback stored metrics in JSON format for post-processing and visualization through TensorBoard. For experimentation, both DQN and PPO models were trained under two configurations to analyze performance sensitivity. In the first configuration, DQN used a smaller batch size (32) with a learning rate of 1×10^{-4}, while PPO used a larger batch size (256) with a learning rate of 2.5×10^{-4}. In the second configuration, the learning rate and replay buffer size (for DQN) were interchanged between the two algorithms. Overrall, experimental procedures would be conducted on DQN and PPO variants, each having 2 different variants of hyperparameter. The final trained models would undergo testing procedures according to their performance, accuracy, and gameplay.

E. Evaluation Protocol

After training, each model was evaluated for 200 episodes under deterministic policy mode to assess generalization performance. The evaluation metrics included:

- Average episodic reward
- Episode length
- Training loss and value loss trends
- Q-value and temporal difference (TD) error for DQN
- Policy entropy, KL divergence, and explained variance for PPO

These indicators provided a consistent basis for assessing convergence speed, training stability, and overall learning efficiency across both algorithms.

F. Evaluation Metrics

To compare both algorithms, the following metrics were analyzed:

- **Average Episodic Reward:** Measures performance over training episodes.
- **Convergence Speed:** Number of episodes required to reach a specific reward threshold (e.g., +15 average reward).
- **Training Stability:** Variance in reward between evaluation intervals.
- **Computation Time:** Total wall-clock time required to achieve near-optimal performance.

IV. Results

This section presents a comparative analysis of the training and evaluation results for DQN and PPO agents across two hyperparameter configurations. Batch 1 used a batch size of 32 with a learning rate of 0.0001, while Batch 2 used a batch size of 256 and a learning rate of 0.00025. Each configuration

was trained under identical preprocessing and environment settings for fair comparison. The training and testing results were logged across 5 million timesteps, where each training observation corresponds to one batch of steps, and each testing observation corresponds to a complete episode.

The following RL variants were implemented:

- **DQN-1**: Batch size 32, learning rate 0.0001, total of 2493 episodes
- **DQN-2**: Batch size 256, learning rate 0.00025, total of 2590 episodes
- **PPO-1**: Batch size 256, learning rate 0.00025, total of 2198 episodes
- **PPO-2**: Batch size 32, learning rate 0.0001, total of 2460 episodes

These episode counts reflect the number of full game rollouts required for each model to complete 5 million training steps. Notably, PPO variants completed training in fewer episodes compared to DQN, suggesting that PPO's on-policy learning leads to faster convergence per episode despite higher sample requirements per update.

A. *Rewards*

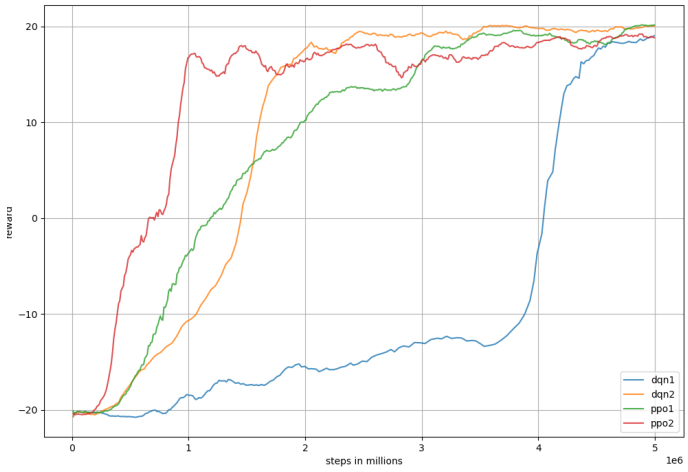

Fig. 1. Rewards Training Phase Result

Figure 1 shows the recorded training reward progression over 5 million timesteps. PPO-2 demonstrated the fastest early-stage convergence, achieving playable performance after approximately one million steps, followed closely by DQN-2, which reached a similar threshold after 1.5 million steps. By the end of training, DQN-1, DQN-2, PPO-1, and PPO-2 achieved mean final rewards of 18.9, 20.03, 20.13, and 18.87, respectively. PPO models generally achieved comparable or higher reward stability with fewer episodes highlighting improved learning efficiency across time.

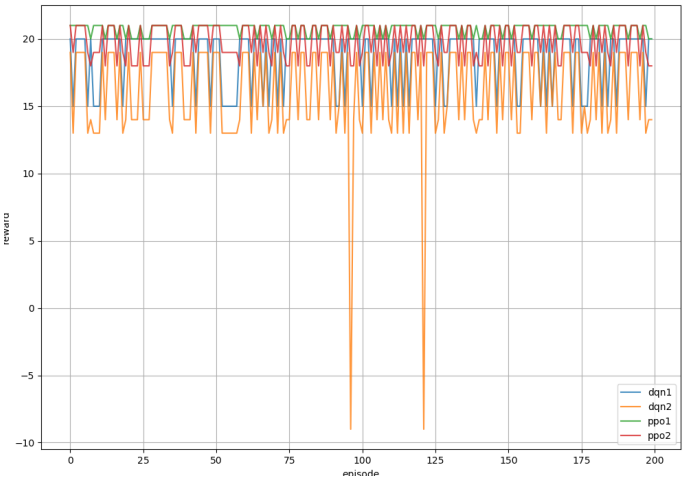

Fig. 2. Rewards Testing Phase Result

Figure 2 presents the testing phase results across 200 episodes. PPO-1 exhibited the highest and most stable rewards, consistently reaching between 20 and 21. PPO-2 followed with slightly wider variability (18–21), while DQN-1 achieved a narrower range (15–20). DQN-2, however, underperformed in several evaluation episodes, recording a minimum reward of -9 and a maximum of 19. Collectively, PPO's reward curves are smoother, indicating more reliable policy generalization and reduced variance during evaluation.

B. *Loss*

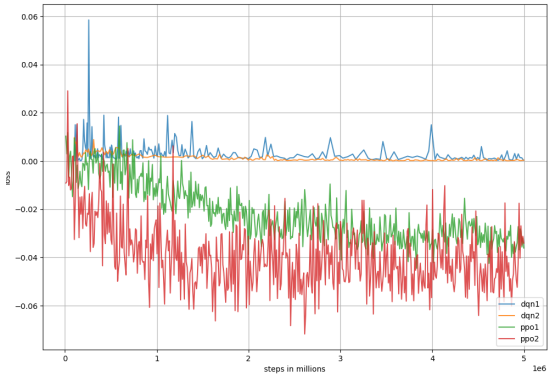

Fig. 3. Loss Training Phase Result

The loss trajectories in Figure 3 further emphasize the structural differences between the algorithms. DQN variants maintained low, stable positive losses between 0.00 and 0.05, reflecting effective control of temporal-difference error through experience replay. DQN-1 exhibited brief spikes near 0.05 around the 30,000th step but quickly returned to stability, confirming robust target network synchronization. PPO variants, in contrast, displayed negative loss values between -0.05 and -0.04, characteristic of policy-gradient optimization. The slightly higher variance in PPO-2's loss indicates more aggressive policy updates, aligning with its faster early reward

convergence. These trends show that DQN favors consistent Q-value refinement, whereas PPO uses dynamic updates to optimize policy performance directly.

C. Episode Length

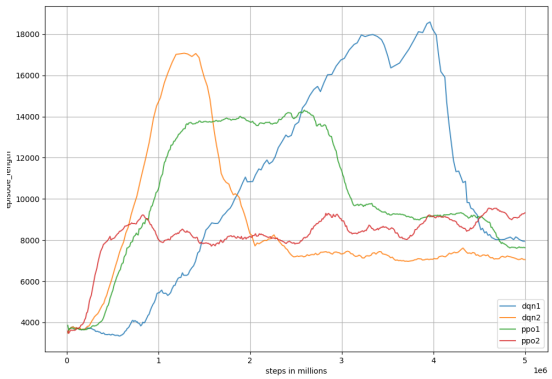

Fig. 4. Episode Length Training Phase Result

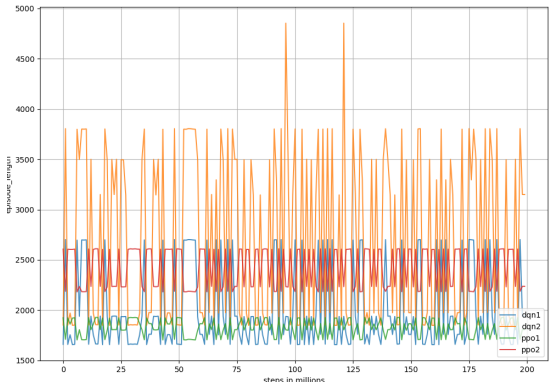

Fig. 5. Episode Length Testing Phase Result

Figures 4 and 5 display the mean episode lengths during training and testing. PPO agents maintained shorter and more consistent episode lengths (8,000–9,500 steps in training; under 2,000 in testing), suggesting more efficient gameplay behavior and faster decision-making. DQN models required longer trajectories peaking above 18,000 steps during training and up to 4,000 in testing reflecting less optimal action selection. Combined with the total episode counts, PPO's ability to complete training in fewer episodes indicates both better policy efficiency and improved sample utilization.

D. DQN Q-Values and TD-Values Comparison

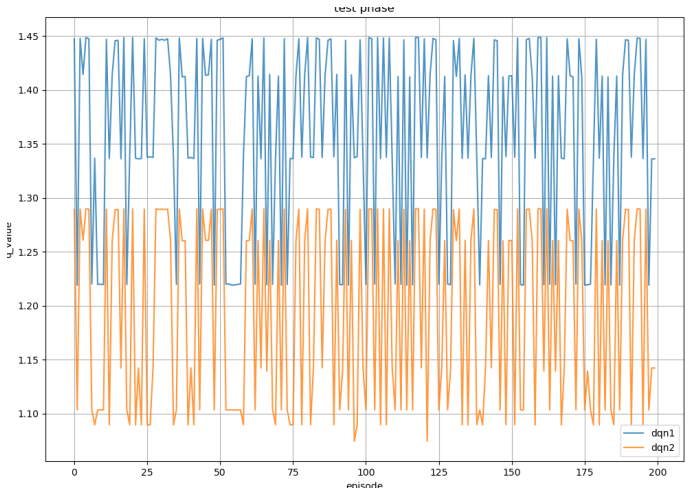

Fig. 6. Q-Value testing phase result

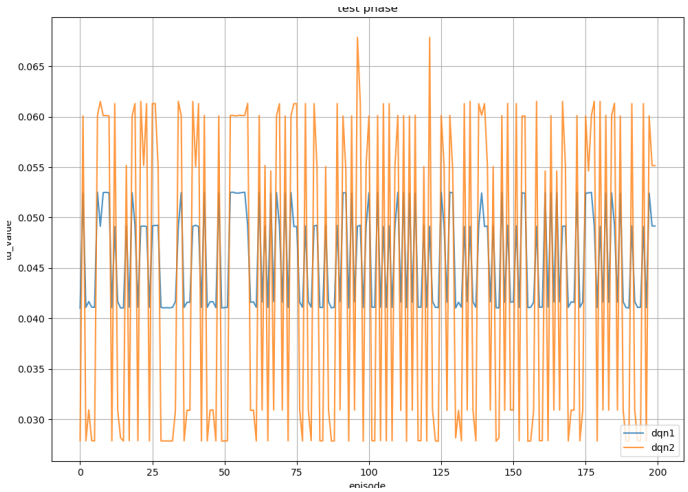

Fig. 7. TD-Value testing phase result

Figures 6 and 7 presents the testing phase Q-value and TD-value distributions for DQN-1 and DQN-2 across 200 episodes. The DQN-1 agent exhibited smoother and more consistent Q-value fluctuations, maintaining an average around 0.045, while DQN-2 displayed significantly higher volatility, with values frequently oscillating between 0.03 and 0.065. This high variance in DQN-2 suggests unstable value estimation and less reliable policy confidence during evaluation. The stability of DQN-1's Q-values indicates a more converged policy and a well-calibrated value network, whereas DQN-2's abrupt TD-value spikes may reflect overestimation bias or inconsistent temporal difference updates. Such instability can lead to erratic action selection and diminished long-term performance reliability. Overall, DQN-1 demonstrated superior temporal consistency and maintained a steady Q-value profile, signifying a more balanced trade-off between exploration and exploitation. In contrast, DQN-2's erratic TD-values imply

sensitivity to state transitions and weaker convergence properties, resulting in less predictable decision-making under test conditions.

E. PPO Variants Results

TABLE II
PPO TRAINING AND TESTING METRICS

Metric	**PPO-1**	**PPO-2**
Training Steps	2.50×10^6	2.50×10^6
Loss	-0.0218	-0.0392
Learning Rate	2.5×10^{-4}	1.0×10^{-4}
Approx. KL Divergence	0.0063	0.0163
Clip Fraction	0.145	0.287
Clip Range	0.1	0.1
Explained Variance	0.857	0.908
Entropy Loss	-1.111	-1.080
Policy Gradient Loss	-0.0103	-0.0198
Value Loss	0.0210	0.0129
Entropy (Testing)	1.063	0.757

Both PPO variants converged successfully, albeit through distinct learning dynamics. PPO-2's lower learning rate yielded higher explained variance (0.908) and smaller value loss (0.0129), indicating stronger critic accuracy. However, its higher KL divergence (0.0163) and clip fraction (0.287) reveal more frequent clipping, suggesting less stable policy updates. PPO-1 maintained smaller KL divergence (0.0063) and higher entropy (1.063), which favored balanced exploration and stable convergence. These results reinforce the trade-off between learning speed and policy robustness observed across PPO variants.

V. DISCUSSION

The experimental outcomes reveal clear distinctions in convergence dynamics between the DQN and PPO families. DQN agents exhibited faster initial reward growth particularly with the larger batch configuration (DQN-2) owing to their off-policy nature and efficient sample reuse from the replay buffer. However, this advantage was offset by higher Q-value volatility and overestimation bias, which led to fluctuating final performance and less stable policy behavior. In contrast, PPO agents displayed slower initial improvement but maintained smoother reward trajectories, shorter episode lengths, and more consistent long-term convergence patterns. These characteristics reflect PPO's clipped objective and on-policy optimization, which inherently favor stability over early acceleration.

The reduced episode counts for PPO (2198 and 2460) compared to DQN (2493 and 2590) indicate greater policy efficiency per training run. Although PPO required more gradient updates per step due to its on-policy structure, it extracted more meaningful information from each environment interaction. This resulted in denser learning updates and faster convergence per episode. Conversely, DQN's reliance on replayed transitions extended the number of episodes required for stable Q-value approximation, emphasizing a trade-off between sample efficiency and per-episode learning density.

It is important to acknowledge that all experiments were conducted under single-seed conditions, meaning the results reflect deterministic trends rather than statistically averaged outcomes. Nonetheless, since all runs shared identical hardware, environment settings, preprocessing pipelines, and logging configurations, the observed performance differences can be confidently attributed to intrinsic algorithmic behavior rather than external computational variability.

VI. CONCLUSION AND FUTURE WORK

This study presented a comparative analysis of DQN and PPO in the Atari Pong environment, emphasizing convergence behavior, reward stability, and training efficiency under two distinct hyperparameter configurations. Both algorithms successfully learned to achieve near-optimal gameplay, but their learning dynamics diverged significantly.

DQN agents, particularly DQN-2, demonstrated faster early-stage improvements due to replay memory reuse and stable gradient updates but required more total episodes (over 2500) to reach convergence. PPO agents, conversely, attained smoother long-term learning with fewer episodes 2198 for PPO-1 and 2460 for PPO-2 indicating higher policy efficiency per interaction cycle. Among these, PPO-1 achieved the best balance between reward consistency, exploration entropy, and convergence speed, while PPO-2 achieved more accurate critic estimations at the cost of reduced exploration diversity.

Overall, PPO outperformed DQN in stability, episode efficiency, and final reward reliability, while DQN remained advantageous in computational simplicity and faster early-stage adaptation. These findings validate the effectiveness of policy-gradient optimization for tasks requiring sustained stability and generalization.

Future research will extend this analysis through multi-seed experiments to assess statistical reliability, incorporate adaptive learning-rate schedules, and evaluate other actor-critic algorithms such as A3C, SAC, and DDPG. Further exploration into runtime profiling and scalability across additional Atari benchmarks will enhance the understanding of algorithmic efficiency in varied RL contexts.

REFERENCES

[1] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," in *NIPS Deep Learning Workshop*, 2013.

[2] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[3] H. P. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, pp. 2094–2100, 2016.

[4] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, D. Silver, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *International Conference on Machine Learning (ICML)*, 2016, pp. 1995–2003.

[5] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver, "Rainbow: Combining improvements in deep reinforcement learning," in *AAAI Conference on Artificial Intelligence*, 2018, pp. 3215–3222.

[6] V. Mnih, A. P. Badia, M. Mirza, A. Harutyunyan, T. Lillicrap, M. Hessel, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *International Conference on Machine Learning (ICML)*, 2016, pp. 1928–1937.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[8] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *AAAI Conference on Artificial Intelligence (AAAI)*, vol. 32, 2018, pp. 3207–3214.